

Task-based control lab

Robot arm control

1 Content of this lab

The goal of this lab is to control one arm of the Baxter robot, first in simulation then on the real robot.

The robot is simulated with Gazebo¹. In the simulation is placed a green sphere. The goal is to move the arm so that the sphere is centered and at a given distance from the camera. The robot control will be performed in C++ and will rely on:

- The ROS framework to handle communication between the simulator and the control program (actually hidden inside the robot class).
- The ViSP library² to manipulate vectors and matrices and to perform linear algebra.

The actual classes that are used are detailed in Appendix A.

1.1 Environment setup

ROS environment should already be set up in the computers. You can setup Qt Creator or VS with the `ros_management_tools`³:

```
# go to the tbc_baxter_vs folder
colbuild -t # compiles this package
ideconf # build the configuration file for Qt Creator and VS code
```

1.2 Structure of the `tbc_baxter_vs` package

The only file to be modified is `main.cpp`, which is the main file for the C++ program.

The simulation can be launched with: `roslaunch tbc_baxter_vs sim.launch`.

We can see that the simulator sends the joint states and the camera image to the control node. In return, the control node sends the joint setpoints to the simulator.

The simulation can be run with a moving ball with: `roslaunch tbc_baxter_vs sim.launch manual:=false`.

1.3 The Baxter robot (arm)

The considered robot is one arm of Baxter. A camera is placed at inside the end-effector of the robot. The robot is controlled through joint velocity $\dot{\mathbf{q}}$. In the initial state of the control node, the error and interaction matrix are not computed, inducing a null joint velocity setpoint.

¹<https://gazebo.org>

²Visual Servoing Platform, <http://visp.inria.fr>

³https://github.com/oKermorgant/ros_management_tools

2 Expected work

2.1 Basic visual servoing

The first goal is to actually compute a classical visual servoing. The features are defined as $\mathbf{s} = (x, y, a)$ where (x, y) is the position of the center of gravity of the sphere and a is its area in the image.

The current features are accessed through `arm.x()`, `arm.y()`, `arm.area()` while the desired ones are set to 0 for (x, y) and accessed through `arm.area_d()` for the area. The error vector $\mathbf{e}$ has to be defined from the current and desired features.

Similarly, the interaction matrix of $\mathbf{s}$ should be defined in `L`. The feature Jacobian, that expresses the time-variation of $\mathbf{s}$ with regards to $\dot{\mathbf{q}}$ should then be computed from `L` and `arm.cameraJacobian()`.

The desired joint velocities can then be computed with $\dot{\mathbf{q}} = -\lambda \mathbf{J}^+ \mathbf{e}$. Of course this control law does not take into account the joint limits.

The gain λ should be equal to `arm.lambda()`, in order to be tuned online.

2.2 Feedforward term

When observing a moving object, the proportional control will always lag behind the object. One way is to increase the gain to have a more reactive control but might get instable. Another way is to try and identify the motion of the object in the image and use a feedforward to compensate. You can implement this part in the `if(use_feedforward)` block.

2.3 Avoiding useless rotations

The sphere-based features allow any rotation around the Z-axis of the camera as it does not change the object in the image. In order to stabilize the system we can add a constraint that is having the X-axis of the camera orthogonal to the Z-axis of the base frame.

The rotation matrix ${}^b\mathbf{R}_c$ can be obtained with:

```
const auto bRc = arm.cameraPose().getRotationMatrix();
```

Knowing that ${}^b\dot{\mathbf{R}}_c = {}^b\mathbf{R}_c [\boldsymbol{\omega}_c]_\times$, express the interaction matrix of this new feature and add it to the control.

2.4 Visual servoing with joint limits

We will use the augmented Jacobian framework⁴. In this framework, the constraints are modeled as additional sensor features. The desired value simply corresponds to the avoidance direction, and can usually be defined by using the center of the valid interval: $\mathbf{q}^* = \frac{1}{2}(\mathbf{q}^+ + \mathbf{q}^-)$. A safe interval $(\mathbf{q}^{s-}, \mathbf{q}^{s+})$ has to be defined with a margin $\rho \in]0., 0.5[$ with:

$$\begin{cases} \mathbf{q}^{s+} = \mathbf{q}^+ - \rho(\mathbf{q}^+ - \mathbf{q}^-) \\ \mathbf{q}^{s-} = \mathbf{q}^- + \rho(\mathbf{q}^+ - \mathbf{q}^-) \end{cases}$$

⁴O. Kermorgant, F. Chaumette. Dealing with constraints in sensor-based robot control. In IEEE Trans. on Robotics, Feb 2014

ρ can be set to e.g. 5 to 10%.

The classical weighting function is depicted in Fig. 1. It is available in the code, as the `h_weight` function:

$$h(s, s_{\text{act}}, s_{\text{max}}) = \begin{cases} h(-s, -s_{\text{act}}, -s_{\text{max}}) & \text{if } s_{\text{act}} \geq s_{\text{max}} \\ 0 & \text{if } s < s_{\text{act}} \\ \frac{s - s_{\text{act}}}{s_{\text{max}} - s} & \text{otherwise} \end{cases}$$

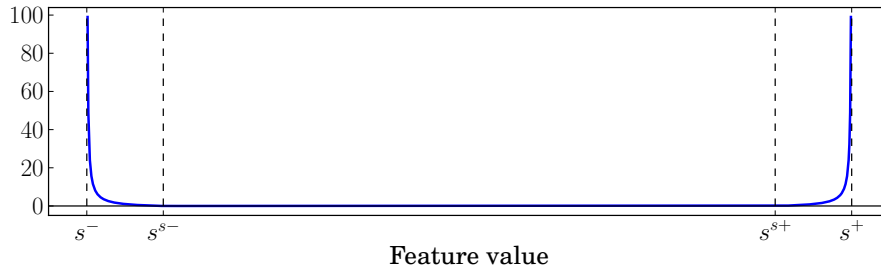


Figure 1: Typical weighting function from $(s^-, s^{s-}, s^{s+}, s^+)$

The control then yields:

$$\dot{\mathbf{q}} = \arg \min \|\mathbf{H}\tilde{\mathbf{J}}\dot{\mathbf{q}} + \lambda\mathbf{H}\tilde{\mathbf{e}}\|^2$$

where: $\mathbf{H} = \begin{bmatrix} \mathbf{I}_4 & 0 \\ 0 & \text{Diag}(h_i) \end{bmatrix}$, $\tilde{\mathbf{J}} = \begin{bmatrix} \mathbf{J} \\ \mathbf{I}_n \end{bmatrix}$ and $\tilde{\mathbf{e}} = \begin{bmatrix} \mathbf{e} \\ \mathbf{q} - \mathbf{q}^* \end{bmatrix}$

The solution to the minimization can be explicitly written as $\dot{\mathbf{q}} = -\lambda(\mathbf{H}\tilde{\mathbf{J}})^+\mathbf{H}\tilde{\mathbf{e}}$.

2.5 Visual servoing and joint control with joint limits

If we use the above formulation then the robot avoids its joint limits but the control law may be discontinuous. Indeed, $(\mathbf{H}\tilde{\mathbf{J}})$ is a 11×7 matrix whose rank may vary from 4 (all joints in their safe zone) to 7 (at least 3 joints close to their limits). A solution is to set arbitrary values for 3 joints.

A good choice is to use the default value (output of `arm.init()`) as desired value for joints 3...7 and keep their weight equal to 1. The joints 1...4 will be used for the visual servoing while avoiding their limits.

2.6 Use on the real robot

If the simulation is working fine, you can control the real robot simply by connecting to the rosmaster of the robot. This is done in a terminal with:

```
ros_baxter
```

A `[baxter]` text should appear in the prompt and for all new terminals afterwards. Then just the Baxter bridge from the same terminal:

```
ros2 run baxter_bridge bridge
```

This node will transfert ROS 2 topics to good old ROS 1 topics that Baxter still uses. You can then run your node from another terminal of the IDE.

```
ros2 run tbc_baxter_vs baxter_vs
```

You can also indicate whether you want to control the left or right arm. The visual detector is designed to detect green objects on the right camera and red objects on the left one. An additional image processing window will open in order to tune the Hue and Saturation thresholds, that may depend on the light conditions of the room.

If you want to go back to a local simulation, you can run the following line:

```
ros_reset
```

It will be valid for all new terminals afterwards.

A Main classes and tools

A.1 ViSP classes

This library includes many tools for linear algebra, especially for 3D transformations. The documentation is found here: <http://visp-doc.inria.fr/doxygen/visp-daily/classes.html>.

The main classes from ViSP (at least in this lab) are:

- **vpFeaturePoint** (variable `p`) represents a Cartesian 2D point as feature. It can be updated with `p.set_xyZ(x,y, 1);`. The main interest is that the corresponding 2×6 interaction matrix can be retrieved by `p.interaction();`
- **vpMatrix** represents a classical matrix, can then be transposed, inversed (or pseudo-inversed), multiplied with a vector, etc.
- **vpColVector** is a column vector with classical mathematical properties.

These class can be declared and used as follows:

```
vpMatrix M(2,6); // matrix of dim. 2x6
M[0][0] = M[1][1] = 1; // element-wise assignment
vpColVector v(6); // column vector of dim. 6
v[0] = 1; // element-wise assignment
M*v; // matrix-vector multiplication
```

Smaller matrices and vectors can be inserted or extracted:

```
vpMatrix M(2,6);
auto Msub = M.extract(0,0,1,2); // extracts the 1st row and 2 first columns, Msub is 1x2
M.insert(Msub, 1,2); // inserts Msub into M, starting at row 1, col 2

vpColVector v(6); // column vector of dim. 6
auto Vsub = v.extract(1,3); // extracts elements at indices (1,2,3)
v.insert(0,Vsub); // inserts Vsub into v at row 0
```